

Summer Internship Project with Buffalo Solar

Presented by,
Hariprakash Karthikeyan

Supervised by,
Ben Luce
Director of Technical Design,
Advocacy & Outreach



Introduction

Over the last two months, we focused on building a user-friendly dashboard to monitor and analyze solar site performance.

The system connects directly to the SolarEdge API and local CPS files to pull production and power data.

We implemented features for visualizing energy trends, detecting outages, and comparing performance across months and years.

Prediction support was added to compare expected vs actual site performance.

The dashboard supports multiple sites, custom thresholds, CSV exports, and interactive graphs for deeper insights.



Tech Stack

- **Python** – Core scripting and data processing
- **Pandas** – Data wrangling and transformation
- **Streamlit** – Web-based interactive dashboard interface
- **Plotly** – Interactive and hoverable visualizations
- **SolarEdge API** – Live data retrieval for 450+ solar sites
- **OpenPyXL / xlrd** – Reading CPS Excel files
- **File System Integration** – Local folder handling for multi-year, multi-site data
- **CSV Support** – Upload, export, and prediction data management



Core Features



- **End-to-End Site Monitoring:** Automatically loads and visualizes production and power data for hundreds of solar sites, with month/year-level granularity.
- **Outage Detection & Custom Thresholds:** Flags underperforming sites using energy-based thresholds, with options to fine-tune outage sensitivity per site.
- **Prediction & Performance Comparison:** Supports uploading forecasted data and overlays it on actual performance to highlight deviations or anomalies.

MELIORA MELIORA MELIORA MELIORA MEI

- ### Solar Dashboard Settings

Select Data Source

☒ SolarEdge

☐ CPS

Select Site

674248 — 4392 Whitehouse Resi... 

☒  Outage Filter

☐ Enable CSV export

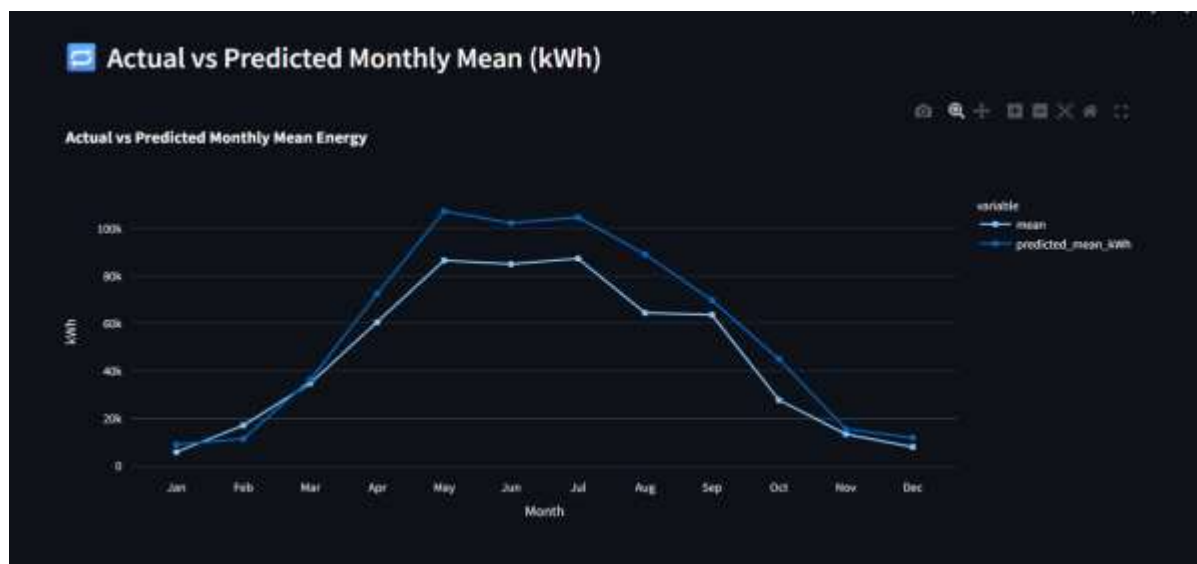
☐ Compare 2 Years

 Upload Prediction CSV(s) 

Drag and drop files here

Limit 200MB per file • CSV, XLSX

[Browse files](#)



MEI IORA MEI IORA MEI IORA MEI IORA MEI

- [illegible]

Interactive Dashboard Features

```
# --- Sidebar Settings ---
with st.sidebar:
    st.header("☁ Solar Dashboard Settings")

    data_source = st.radio("Select Data Source", ["solarEdge", "CPS"])

    if d (variable) site_id: Any
        site_id = st.selectbox("Select site", list(site_map.keys()), format_func=lambda x: f"{x} - {site_map[x]}")
    else: # CPS
        cps_site_id = st.selectbox("Select CPS Site", list(cps_site_map.keys()))

    app_func = "sum"

    filter_outage = st.toggle("🔍 Outage Filter", value=True)
    if filter_outage:
        threshold = st.slider("Outage Ratio Threshold", 0.1, 0.7, 0.3)
    else:
        threshold = None

    enable_csv = st.checkbox("Enable CSV export")
    compare_years = st.checkbox("Compare 2 Years")

    uploaded_pred_files = st.file_uploader(
        "📁 Upload Prediction CSV(s)", type=["csv", "xlsx"], accept_multiple_files=True,
        help="Each file must contain: month_name, year, predicted_mean_kwh"
```

- Site Selection & Filtering:** Dropdown menus to select individual or multiple sites for comparison.
- Custom Controls:** Users can toggle between raw/mean data, select year/month ranges, and adjust outage thresholds.
- CSV Export Options:** Graph data and predictions can be downloaded for offline analysis or reporting.



CPS Data Support

- Added support for CPS data stored locally in Excel files.
- Each site has multiple yearly files, dynamically loaded and combined.
- Allows consistent handling alongside SolarEdge API data.

```
# --- Read all yearly files from selected CPS site folder ---
def load_cps_data(folder_path: Path) -> pd.DataFrame:
    all_data = []
    for file in folder_path.glob("*.xlsx"):
        try:
            df = pd.read_excel(file)
            df = df.rename(columns={"time": "datetime", "Power generation (kWh)": "energy_kwh"})
            df["datetime"] = pd.to_datetime(df["datetime"])
            df["month"] = df["datetime"].dt.month
            df["year"] = df["datetime"].dt.year
            df["month_name"] = df["datetime"].dt.month.map(lambda m: calendar.month_abbr[m])
            all_data.append(df[["datetime", "month", "year", "month_name", "energy_kwh"]])
        except Exception as e:
            st.warning(f"⚠ Could not read {file.name}: {e}")
    if not all_data:
        return pd.DataFrame()
    return pd.concat(all_data, ignore_index=True)
```

MELIORA MELIORA MELIORA MELIORA MELIORA MELIORA

- MEI

Data Pipeline Overview

- **Data Sources:**
 - **SolarEdge API** – Live data fetched per site
 - **CPS Excel Files** – Local data parsed from yearly .xlsx files
- **Processing:**
 - Cleaned, aggregated, and merged using pandas
 - Handles gaps, filters outages, computes mean and variance
- **Visualization:**
 - Displayed via Streamlit with Plotly charts
 - Supports interaction, prediction overlays, and export features



MELIORA MELIORA MELIORA MELIORA MELIORA

- MELIORA MELIORA MELIORA MELIORA MELIORA MELIORA



Challenges Faced

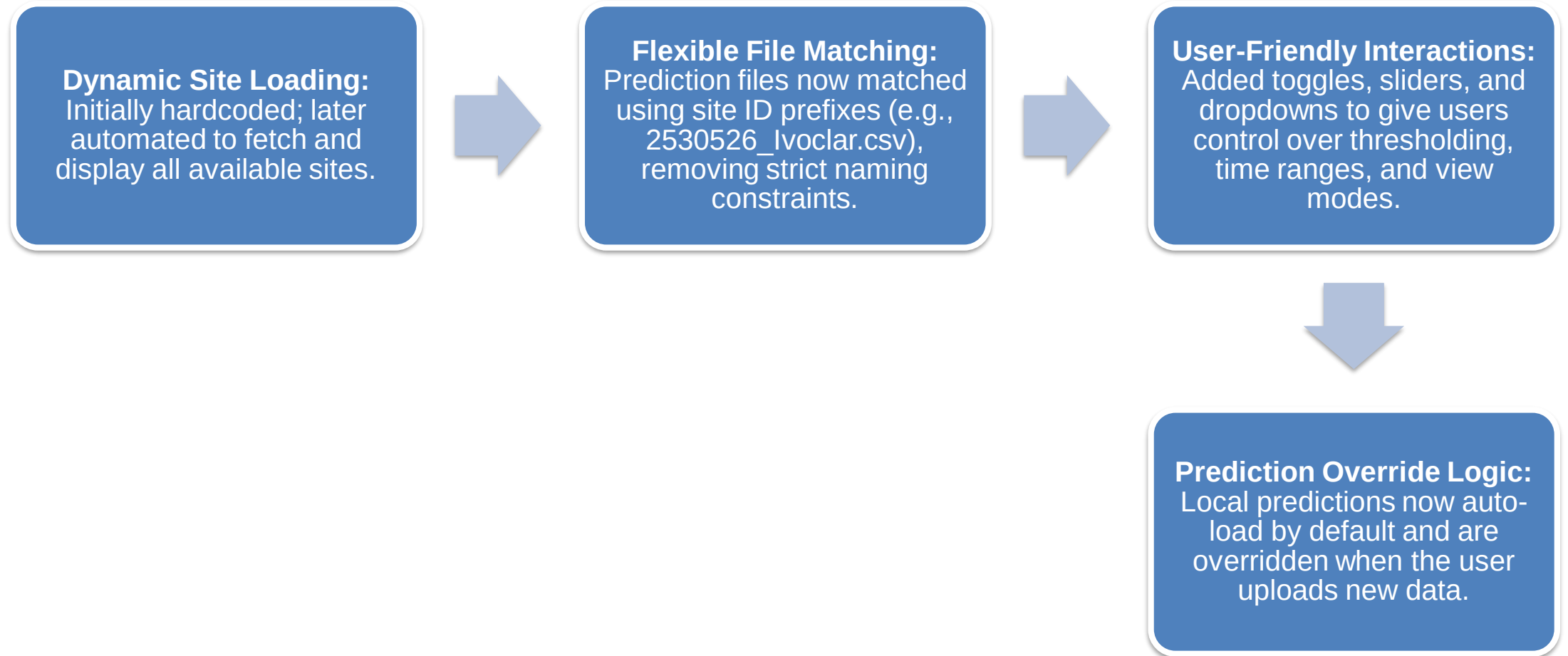
- **API Rate Limits & Failures:** Had to manage delays and retries while fetching large-scale data from SolarEdge.
- **Inconsistent CPS File Structures:** Yearly files differed in format, requiring flexible and defensive parsing logic.
- **Missing or Sparse Data:** Some sites lacked complete data, so graphs and calculations had to handle gaps gracefully.
- **Multi-Site Scalability:** Designing the dashboard to support hundreds of sites without slowing down performance.

How We Handled It:

- Used caching and modular functions to streamline data loading.
- Built file-reading logic to auto-detect formats and merge multiple years.
- Integrated fallback conditions for graphs when data is missing.



Improvements Made



Future Scope

Notify	Automated Alerts: Notify users when sites underperform or have frequent outages.
Add	User Authentication & Upload Portal: Add secure login and site-specific upload functionality for predictions or manual data.
Forecasting	Forecasting Integration: Train and embed ML models to generate predictions from historical data.
Reporting	Performance Reporting: Generate downloadable PDF reports summarizing site health and energy performance.
Improve	Mobile Optimization: Improve dashboard layout and responsiveness for mobile or tablet users.



MELIORA MELIORA MELIORA MELIORA MELIORA



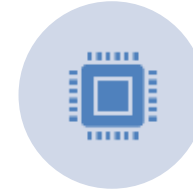
Summary & Key Takeaways



Built a scalable, interactive dashboard to monitor solar site performance across multiple data sources.



Integrated outage detection,
prediction comparison, and
multi-site support.



Created a flexible system that adapts to varying data formats, user inputs, and site configurations.



Key Impact:



Simplified large-scale solar data exploration and decision-making.



Made performance monitoring more visual, accessible, and actionable for end users.

Thank
You!

